Eseményvezérelt alkalmazások: 6. gyakorlat

A munkafüzetben megismerkedünk a többszálú programozással és az aszinkron (párhuzamos) műveletek megvalósításával. Most a *DocumentumStatistics* esetében a fájlbetöltést bővítjük ki, hogy egyszerre több szöveges állomány beolvasását és elemzését is elvégezhessük. Az eddigi felület fő részét (szöveges mezők, statisztikák) átvisszük egy egyedi vezérlőbe. Ebből minden beolvasott fájl után létrehozunk egyet, majd a főablakhoz adjuk egy `TabControl` vezérlőbe.

1 Több fájl megnyitása lapokkal ^{KM}

Egészítsük ki a meglevő `DocuStatView` projektet, hogy egyszerre több fájlt is be tudjon olvasni, és ezeket külön lapokon meg tudja jeleníteni!

1.1 UI kiszervezése

Első lépésként szervezzük ki a korábban létrehozott felhasználói felület menü alatti részét saját vezérlőbe. Adjunk hozzá a projekthez egy új `UserControl`-t, legyen ennek a neve `DocuStatControl`. A felület egyszerű reprodukálásához célszerű lehet a meglevő elemeket a `DocuStatDialog`-ról átmásolni és beilleszteni, így a korábban beállított property-k megmaradnak. Az így elkészült `DocuStatControl` a lentebbi ábrán látható.

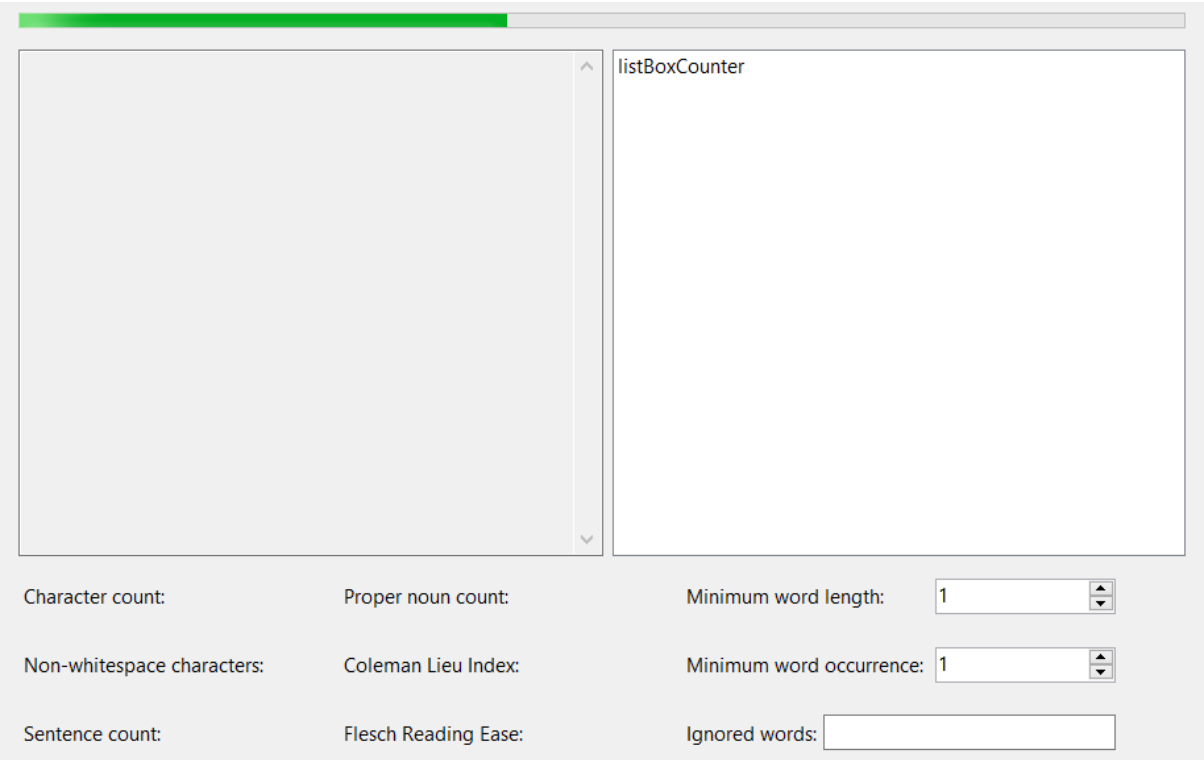


Figure 1: A létrehozott UserControl

Szervezzük ki a már meglevő felületet kezelő kódot is az alábbi osztálydiagramon látható módon.

- A `LoadFile` a kapott `IFileManager`-t felhasználva elvégzi a példányosítást és feliratkozik a modell eseményeire az `UpdateFileContent`, `UpdateTextStatistics` és `UpdateWordList` eseménykezelőkkel. Végül elvégzi a betöltést is. Itt ne végezzünk most hibakezelést, majd a hívó oldalon fogjuk a kivételeket elkapni.
- Az `UpdateFileContent` a szövegdoboz kitöltését végzi, az `UpdateTextStatistics` a statisztika címkéket tölti ki a kapott `TextStatisticsReadyEventArgs` alapján, az `UpdateWordList` pedig a jobb oldali listát építi fel a modell `FilteredWordCount` tulajdonságából.
- Az `UpdateWordFilter` a három szűrő vezérlő (`spinBoxMinLength`, `spinBoxMinOccurrence`, `textBoxIgnoredWords`) értékét vezeti át a modellbe; ezek eseményeire a konstruktorban iratkozunk fel.
- A publikus `ResetFilter` metódus a szűrő vezérlőit állítja vissza az alapértelmezett értékükre; ezt fogja a főablak menüpontja meghívni.

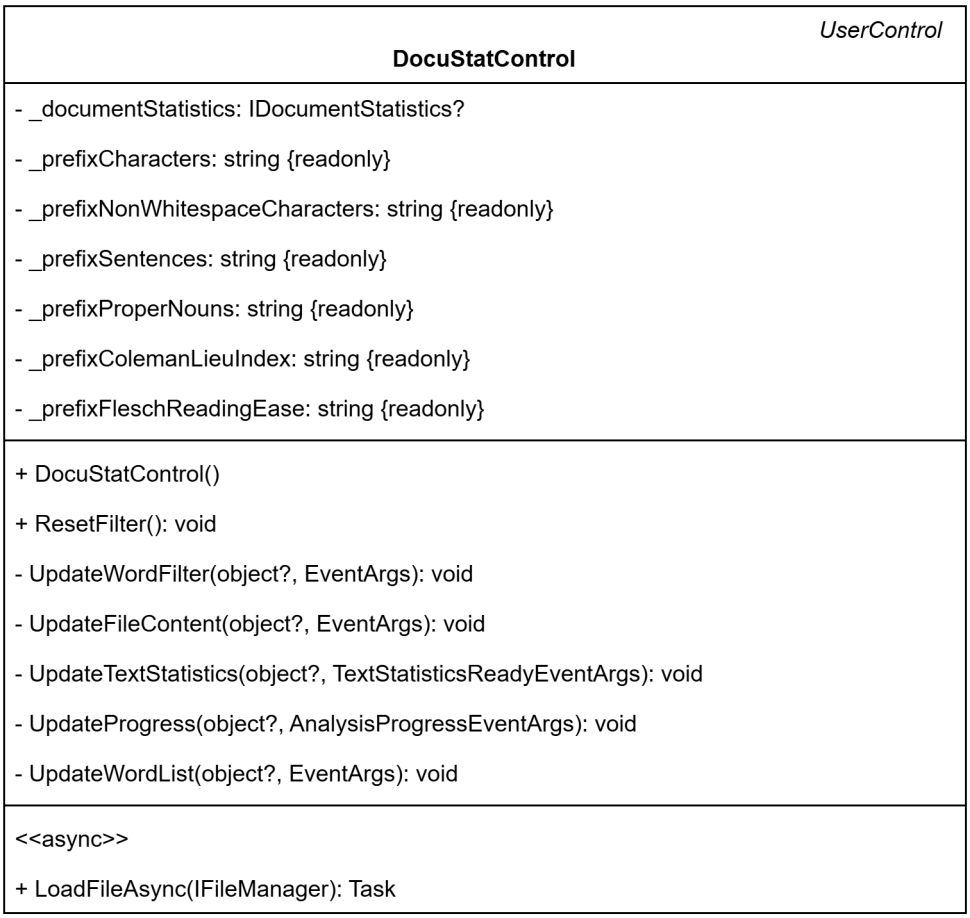


Figure 2: A DocuStatControl osztálydiagramja

1.2 Lapok létrehozása

Miután kiszerveztük a megjelenítést, töröljük le a `DocuStatDialog`-ról ezeket a vezérlőket, és vegyünk fel helyettük egy `TabControl`-t. A `(Name)` property legyen `tabControl`, a `Dock` legyen `Fill`, és töröljük az alapértelmezetten létrehozott `tabPage`-eket!

Kód oldalon módosítsuk az `OpenDialog` metódust, hogy az `OpenFileDialog` dialógus ablak `Multiselect` property-je igaz legyen, ez lehetővé teszi, hogy a felugró ablakban több fájlt is megadhassunk. Az így kapott útvonalakat a `FileNames` property-n keresztül érjük el (mint `string` tömb). A már megnyitott lapok egyszerű kezelése végett töröljük az eddigi tabokat a `tabControl.TabPages.Clear()` segítségével!

A lapok (*tabok*) létrehozásához vegyük fel az `AddTabPage` segédeljárást, ez a paraméterül kapott fájlútvonalhoz létrehoz egy `IFileManager`-t, majd ezzel példányosít egy `DocuStatControl`-t és meghívja a `LoadFile`-t. Hozzunk itt létre egy `TabPage`-et is, aminek a `Controls` gyűjteményéhez adjuk hozzá a létrehozott `DocuStatControl`-t! A `TabPage`-nek célszerű azonosító címkét megadni, ezt akár a konstruktorában, akár a `Text` property-n keresztül tehetjük meg, ez lehet például a fájl neve, amit a kapott teljes útvonalból a `System.IO.Path.GetFileName` segítségével állíthatunk elő. Végezetül a `tabControl.TabPages` gyűjteményéhez a létrehozott `TabPage`-et hozzáadva megjelenik a felületen az új lap.

Itt kezeljük a korábbihoz hasonló módon a lehetséges kivételeket is, hiszen a `FileManagerFactory` és a `LoadFile` is hibát válthat ki, ebben az esetben térjünk is vissza, hogy ne kerüljön fel tényleges lap a felületre.

```
private void AddTabPage(string fileName)
{
    IFileManager? fileManager = FileManagerFactory.CreateForPath(fileName);
    if (fileManager == null)
    {
        // ...
        return;
    }

    try
    {
        DocuStatControl control = new DocuStatControl();
        control.LoadFile(fileManager);
        TabPage tabPage = new TabPage(System.IO.Path.GetFileName(fileName));
        tabPage.Controls.Add(control);
        tabControl.TabPages.Add(tabPage);
    }
    catch (FileManagerException ex)
    {
        // ...
        return;
    }
}
```

A `resetFilterMenuItem`-hez tartozó `ResetFilter` eseménykezelőt alakítsuk át úgy, hogy az aktuálisan megnyitott laphoz (`SelectedTab`) tartozó `DocuStatControl` `ResetFilter` metódusát hívja meg, nem megfeledkezve arról, hogy lehet, hogy nincsen lap kiválasztva, amit `SelectedTab == null` jelez. A `DocuStatControl` elővételéhez használjuk fel, hogy mi állítottuk össze a lapot, és tudjuk, hogy a `DocuStatControl` típusú elem az egyetlen vezérlő benne.

```
(tabControl.SelectedTab.Controls[0] as DocuStatControl)!.ResetFilter();
```

2 Párhuzamosítás

Az eddigi átalakítások után egyszerre több fájlt is meg tudunk nyitni, azonban ezek egymás után, sorrendben fognak betöltődni, és hosszú szöveg esetén a töltés alatt a felület is elveszíti a reszponzivitását, "lefagy". A probléma orvoslására használjuk fel a C# aszinkron programozási lehetőségeit!

Az *async* / *await* minta lényege, hogy háttérszálakon el tudjuk párhuzamosan végezni az időigényes műveleteket `async` hívások segítségével, majd amikor szükségünk van az előállított eredményekre, az `await` operátor használatával meg tudjuk várni ezek befejeződését, mindezt olyan módon, hogy közben a hívást végző folyamat nem "blokkolódik", tehát a felhasználói felület továbbra is kattintható és reszponzív marad.

A használatnak egy alapvető eleme a `Task<T>` osztály, ami egy `T` típusú eredményt előállító aszinkron folyamatot reprezentál (`void` esetén egyszerűen `Task`).

2.1 Perzisztencia és modell párhuzamosítása ^{KM}

A teljes alkalmazáslogikában a fájlok tartalmának betöltése az egyik szűk keresztmetszet hatékonyság szempontjából: egyrészt az I/O műveletek lassú volta miatt (memóriakezeléshez viszonyítva), másrészt a PDF fájlok feldolgozása is időigényes feladat lehet.

Alakítsuk át ezért először az `IFileManager` interfész és implementációinak `Load` metódusát. Az `async` kulcsszó az `await` operátor használatát engedélyezi a függvényen belül, a visszakapott `Task`-on keresztül pedig majd a művelet befejeződését tudjuk megvárni. A függvény aszinkron mivoltát elnevezési konvenció szerint `Async` utótaggal szokás jelezni. A metódus új szignatúrája legyen a következő (az interfészben az `async` kulcsszót ne írjuk ki):

```
public async Task<string> LoadAsync()
{
    // ...
}
```

- Szöveges (TXT) fájl feldolgozása esetén a `File` osztály már támogatja egy fájl tartalmának aszinkron betöltését, használhatjuk a `ReadAllTextAsync` eljárást ehhez.
- PDF fájlok esetén a felhasznált *iText* szoftverkönyvtár nem rendelkezik aszinkron támogatással, de az erőforrásigényes oldalbetöltési műveletet egyszerűen azzá tehetjük, ha egy `Task`-ba foglaljuk. Ezt a `Task.Run` alkalmazásával egyszerűen megtehetjük, ez a paraméterül kapott lambda kifejezés végrehajtását elkezdi aszinkron módon, és visszatér az ezt reprezentáló `Task` objektummal.

```
return await Task.Run(() =>
{
    StringBuilder text = new StringBuilder();
    for (int i = 1; i <= document.GetNumberOfPages(); i++)
    {
        PdfPage page = document.GetPage(i);
        text.Append(PdfTextExtractor.GetTextFromPage(page));
    }
    return text.ToString();
});
```

> *Megjegyzés:* az iText dokumentációja szerint egyszerre csak egyetlen szál férhet hozzá ugyanahhoz a `PdfDocument` objektumhoz. Ezért több oldal párhuzamos feldolgozásával nem foglalkozunk most.

A `DocumentStatistics` modell vonatkozásában a `Load` eljárást tegyük aszinkronná, és kezeljük ennek megfelelően a perzisztenciát is. Mivel a szöveg metrikák előállítása számításigényes, nagyobb fájlokra akár több hosszabb időt (több másodpercet) is igénybe vehet, ezt is háttérfolyamatként, külön `Task`-ban futtassuk.

```
public async Task LoadAsync()
{
    FileContent = await _fileManager.LoadAsync();

    await Task.Run(() =>
    {
        // ...
    });
}
```

2.2 Nézet párhuzamosítása ^{EM}

A nézet rétegben minden `DocuStatControl` vezérlő saját modellel rendelkezik, amelyet be kell töltenie. Tegyük ezt aszinkron lehetővé, a `LoadFile` metódusának refaktorálásával:

```
public async Task LoadFileAsync(IFileManager fileManager)
{
    _documentStatistics = new DocumentStatistics(fileManager);
    _documentStatistics.FileContentReady += UpdateFileContent;
    _documentStatistics.TextStatisticsReady += UpdateTextStatistics;
    _documentStatistics.FilteredWordCountChanged += UpdateWordList;

    // Az új modellnek át kell vennie a felületen már beállított szűrőt.
    UpdateWordFilter(this, EventArgs.Empty);

    await _documentStatistics.LoadAsync();
}
```

Így már a `DocuStatDialog` ablak `AddTabPage` metódusát is átalakíthatjuk, hogy egy új `DocuStatControl` vezérlő felvétele aszinkron lehetséges legyen. A metódus új szignatúrája legyen a következő:

```
private async Task AddTabPageAsync(string fileName)
{
    // ...
}
```

A folyamatban a `DocuStatControl` vezérlő elkészítése és betöltésének párhuzamosított megkezdése után folytathatjuk a tablap összerakását, majd amikor már nincs más teendőnk, a kapott `Task`-ra `await`-et hívva megvárjuk a feladat befejezését. Magát a betöltés megkezdését és befejeztét jelezhetjük például a `TabPage` szövegének módosításával.

```
// ...
DocuStatControl control = new DocuStatControl();
Task loadTask = control.LoadFileAsync(fileManager);
TabPage tabPage = new TabPage("Loading...");
tabPage.Controls.Add(control);
tabControl.TabPages.Add(tabPage);

await loadTask;
tabPage.Text = System.IO.Path.GetFileName(fileName);
// ...
```

Legyen az `AddTabPageAsync` hívását végző `OpenDialog` is aszinkron! Ide is tegyük ki a `private` után az `async` kulcsszót, de a visszatérési érték maradjon `void`, mivel az eseménykezelő típusa megköveteli ezt.

Megtehetnénk, hogy egyszerűen `await AddTabPageAsync(fileName)` hívással megvárjuk az aktuális fájl betöltődését, és addig nem kezdjük el a többit, viszont ezzel továbbra sem használnánk ki a párhuzamosság adta lehetőségeket, ehelyett jobb megközelítés, ha egy `List<Task>` konténerbe először eltesszük ezeket a feladatokat, ezzel elindítva mindet, majd utána várjuk be az összeset. Ez utóbbit végezzük a `Task.WhenAll(tasks)` statikus metódussal, ami visszaadja az összes kapott `Task` befejezettségét jelző `Task`-ot, ezt `await`-elve tudjuk megvárni az összes lap betöltését.

```
// ...
    List<Task> tasks = new List<Task>();
    foreach (string fileName in openFileDialog.FileNames)
    {
        tasks.Add(AddTabPageAsync(fileName));
    }
    await Task.WhenAll(tasks);
// ...
```

Innentől kezdve a lapok betöltése közben továbbra is használható lesz a felület, például láthatjuk a már betöltött tabokat, viszont ebben a köztes állapotban az újabb betöltés indítása, vagy a `ResetFilter` hívása egy még befejezetlen tabon hibát okozhat. Emiatt az eseménykezelőkhöz tartozó menüpontokat (`openFileDialogMenuItem`, `resetFilterMenuItem`) a betöltés idejére tegyük inaktívvá az `Enabled` property beállításával, majd pedig a végén kapcsoljuk ezeket vissza!

2.3 UI frissítés háttérszálról ^{EM}

Grafikus asztali alkalmazásoknál (Windows Forms és majd WPF egyaránt) minden felületi vezérlőt csak az őt létrehozó szál kezelhet, különben *cross-thread operation* miatti kivételt kaphatunk (`InvalidOperationException`). Ezt megoldandó minden `Control` objektum rendelkezik egy `Invoke` metódussal, ami a paraméterül kapott lambda kifejezést a felületet birtokló szálon hajtja végre. Ha nem fontos a háttérszál blokkolása és a frissítés megvárása, használhatjuk a `BeginInvoke` eljárást is.

Az aszinkron metódusok (`async`) szinkronban futnak, amíg el nem érik az első várakozási kifejezést (`await`), utána ez már nem biztosított. Aszinkron végrehajtásra kerül sor akkor is, amennyiben explicit egy új *taszkot* indítunk el, pl. egy `Task` objektum `Start()` metódusával, vagy a `Task.Run()`, vagy a `Task.Factory.StartNew()` meghívásával.

A gyakori manuális szinkronizáció elkerülése érdekében a .NET alkalmazások rendelkezhetnek egy szinkronizációs kontextussal (`SynchronizationContext.Current`), amelynek megadásával implicit szinkronizáció követelhető meg a várakozási kifejezések (`await`) után a hívó szállal. Windows Forms alkalmazások rendelkeznek is egy alapértelmezett szinkronizációs kontextussal, a `SynchronizationContext.Current` értéke egy `WindowsFormsSynchronizationContext` típusú objektum, amely implementálja, hogy `await` utasítás után a UI szálra visszatérünk a `BeginInvoke` használatával.

Példának az `AddTabPageAsync` műveletet vehetjük, ahol a Windows Forms alapértelmezett szinkronizációs kontextusának köszönhetően az `await loadTask` után garantáltan a UI szálon folytatódik a végrehajtás, külön szinkronizáció nélkül is.

```
private async Task AddTabPageAsync(string fileName)
{
    // ...
    try
    {
        DocuStatControl control = new DocuStatControl();
        Task loadTask = control.LoadFileAsync(fileManager);
        TabPage tabPage = new TabPage("Loading...");
        tabPage.Controls.Add(control);
        tabControl.TabPages.Add(tabPage);

        await loadTask;
        // A Windows Forms alapértelmezett szinkronizációs kontextusának köszönhetően
        // garantáltan a UI szálon folytatódik a végrehajtás.

        tabPage.Text = Path.GetFileName(fileName);
    }
    // ...
}
```

Nem ennyire egyértelmű a helyzet a `DocumentStatistics` modell eseményeinél, amelyeket az aszinkron `LoadAsync()` eljárás vált ki. Itt az `InvokeRequired` és a `BeginInvoke()` használatával gondoskodhatunk a szinkronizációról. Például:

```
private void UpdateTextStatistics(object? sender, TextStatisticsReadyEventArgs e)
{
    if (InvokeRequired)
    {
        BeginInvoke(() => UpdateTextStatistics(sender, e));
        return;
    }

    // ...
}
```

Valójában csak a `TextStatisticsReady` kerül háttérszálon kiváltásra, hiszen az a modellben egy explicit elindított *taszkon* belül keletkezik. A `FileContentReady` és a `FilteredWordCountChanged` ezzel szemben a `Task.Run` blokkon kívül, a UI szálon kerül kiváltásra; ott az `InvokeRequired` vizsgálata felesleges – de

nem okoz bajt sem.

> *Tipp:* mivel végső soron a tényleges megjelenítést muszáj ugyanannak a szálnak végeznie, ha például egy nagyon hosszú szöveges fájlt próbálunk betölteni, a teljes tartalom megjelenítése `TextBox`-ban hosszú időt igénybe tud venni, ami alatt az alkalmazás irreszponzív. Ezen minimális munkával egészen sokat tud gyorsítani, ha a szövegdoboz `WordWrap` property-jét hamisra állítjuk, ezzel nem számolódnak ki az automatikus sortörések.

> *Megjegyzés*: A tabok betöltésének párhuzamosításához használhatjuk a `Parallel.ForEach` metódust is. Ebben az esetben minden ciklus kör egy új szálon fog futni. A Parallel osztály biztosítja számunkra, hogy az indított szálak mindegyike végrehajtódjon, mielőtt a futás tovább folytatódik. Mivel a túl sok szál indítása szintén le tudja terhelni a CPU-t, érdemes korlátozni a párhuzamos szálak számát. Ezt a `MaxDegreeOfParallelism` property beállításával tudjuk szabályozni.

3 Feldolgozás előrehaladásának jelzése ^{OP}

Egy több másodpercig futó műveletnél a felhasználót tájékoztatnunk kell arról, hogy hol tart a feldolgozás. Ezt az információt a nézet **nem tudja lekérdezni**: a modell belső ciklusáról csak maga a modell tud. Ez az eseményvezérlés egy újabb, az eddigiektől eltérő jellegű esete: itt az esemény nem egy állapot *végét*, hanem egy folyamat *előrehaladását* jelzi, és ezért nem egyszer, hanem sokszor, gyors egymásutánban kerülhet kiváltásra.

3.1 A modell kiegészítése

Vegyünk fel egy újabb egyedi eseményargumentum osztályt a `Model` könyvtárba `AnalysisProgressEventArgs` néven, egyetlen, csak olvasható `Percentage` tulajdonsággal (0 és 100 közötti érték), majd a hozzá tartozó eseményt és a kiváltásáért felelős privát metódust:

```
public event EventHandler<AnalysisProgressEventArgs>? AnalysisProgressChanged;

// ...

private void OnAnalysisProgressChanged(int percentage)
{
    AnalysisProgressChanged?.Invoke(this, new AnalysisProgressEventArgs(percentage));
}
```

Ne feledjük az eseményt az `IDocumentStatistics` interfészbe is felvenni!

A haladást a legköltségesebb ciklusból, a `ComputeDistinctWords` szavakon végigmenő ciklusából jelentsük. Fontos, hogy ne minden egyes iterációban tegyük ezt: az esemény kiváltása (és a nyomában a felület frissítése) többe kerülne, mint maga az elvégzett munka. Elég századonként jelezni:

```
int step = Math.Max(words.Length / 100, 1);

for (int i = 0; i < words.Length; ++i)
{
    if (i % step == 0)
    {
        OnAnalysisProgressChanged(100 * i / words.Length);
    }

    // ...
}
```

A `LoadAsync` `Task.Run` blokkjának végén, közvetlenül az `OnTextStatisticsReady()` hívása előtt jelentsük a 100%-os készültséget is.

3.2 A nézet kiegészítése

A haladás megjelenítéséhez a Designerben helyezzünk el a `DocuStatControl` tetején, a szövegdoboz fölött egy `ProgressBar` vezérlőt! A `(Name)` tulajdonsága legyen `progressBar`, a `Visible` tulajdonsága pedig hamis, hiszen csak a betöltés idejére szeretnénk megmutatni.

A `LoadFileAsync` metódusban iratkozzunk fel az új eseményre, és a betöltés idejére tegyük láthatóvá a sávot:

```
public async Task LoadFileAsync(IFileManager fileManager)
{
    // ...
    _documentStatistics.AnalysisProgressChanged += UpdateProgress;

    // ...

    progressBar.Visible = true;
    await _documentStatistics.LoadAsync();
    progressBar.Visible = false;
}
```

Az eseménykezelő mindössze a sáv értékét állítja be az eseményargumentumból. Mivel az `AnalysisProgressChanged` a modellben elindított *taszkon* belül keletkezik, itt is szükség van az előző szakaszban megismert szinkronizációra – sőt, ez a legszemléletesebb esete: ez az esemény biztosan háttérszálról, ráadásul sokszor, gyors egymásutánban kerül kiváltásra.

```
private void UpdateProgress(object? sender, AnalysisProgressEventArgs e)
{
    if (InvokeRequired)
    {
        BeginInvoke(() => UpdateProgress(sender, e));
        return;
    }

    progressBar.Value = e.Percentage;
}
```

Kitekintés: a .NET a haladás jelentésére egy külön absztrakciót is kínál, az `IProgress<T>` interfészt és annak `Progress<T>` megvalósítását. Ez utóbbi a létrehozásakor eltárolja az aktuális `SynchronizationContext`-et, és a `Report()` hívásokat magától azon keresztül továbbítja – így a `Progress<T>` használatakor nincs szükség kézi `Invoke`-ra. Saját eseménnyel viszont, mint most az `AnalysisProgressChanged` esetében, a szinkronizáció a mi felelősségünk.

Megjegyzés: több fájlt egyszerre megnyitva a lapok haladásjelzői párhuzamosan mozognak, jól láthatóvá téve, hogy a feldolgozás valóban egyszerre több szálon zajlik.

4 Kitekintés: fájlváltozás figyelése ^{OP}

Az eddigi modell-események mindegyike végső soron a felhasználó egy műveletéből fakadt: megnyitott egy fájlt, tekert a spinboxon, gépelt. Érdemes viszont végiggondolni azt az esetet is, amikor a változás **az alkalmazáson kívülről** érkezik: a megnyitott fájlt egy másik program írja felül, mire a modell magától újratölti és újraelemzi. Ez az az eset, ahol a nézet semmiképp nem tudná lekérdezéssel pótolni az eseményt, hiszen azt sem tudja, mikor kellene kérdeznie.

A megvalósítás vázlata:

1. A perzisztencia rétegben az `IFileManager` interfész kap egy `event EventHandler? ContentChanged` eseményt, a megvalósításai pedig a `System.IO` névtér `FileSystemWatcher` osztályával figyelik a fájlt. A figyelő erőforrást tart életben, ezért az interfésznek `IDisposable`-nek is kell lennie.
2. A modell a konstruktorában feliratkozik erre az eseményre, és a kezelőjében újra meghívja a `LoadAsync()` metódust. Innentől a nézet **változtatás nélkül** frissül: a `LoadAsync` a megszokott eseményeket váltja ki. Épp ezért volt érdemes az eseményeket a *hatásukról* elnevezni – a nézetet nem érdekli, hogy a lista azért változott, mert a felhasználó szűrt, vagy mert a fájl megváltozott a lemezen.

A feladat potenciális buktatói:

- A `FileSystemWatcher` **háttérszálról** jelez, ezért az így indított újratöltés egésze háttérszálon fut – nem csak a `Task.Run` blokkja. Az `InvokeRequired` szerinti szinkronizációt tehát a többi eseménykezelőre is ki kell terjeszteni.
- A szerkesztőprogramok a fájlt több lépésben írják ki, ezért egyetlen mentés is értesítések sorozatát váltja ki – sőt, egyes szerkesztő alkalmazások ideiglenes fájlba írnak, majd átnevezéssel cserélik le az eredetit, ami újabb értesítéseket jelent. Kezeletlenül tehát minden mentés több teljes újratöltést és újraelemzést indítana. A szokásos megoldás egy rövid késleltetés (*debounce*): minden értesítés újraindít egy időzítőt, és csak akkor jelzünk, ha az értesítések elálltak.
- A késleltetés lejártakor is előfordulhat, hogy az író program még zárolja a fájlt, ilyenkor a beolvasás `IOException` kivételt válthat ki. A *debounce* ezt csak valószínűtlenné teszi, kizárni nem tudja, ezért kivételkezelés mellett a betöltést néhányszor, rövid szünetekkel érdemes újrapróbálni.
- Az eseménykezelő `async void` lesz, hiszen az eseménynek nincs `Task` visszatérési értéke. Ezért itt **kötelező** a kivételkezelés: egy `async void` metódusból kiszabaduló kivételt már nem lehet elkapni, az egész alkalmazás terminál tőle.

Megjegyzés: A zárolás **nem** kerülhető meg azzal, hogy a fájlt `FileShare.ReadWrite` megosztással nyitjuk meg. A megosztási mód egyeztetés kérdése: ha az író program kizárólagosan nyitotta meg a fájlt, a mi kérésünk ugyanúgy elbukik. Ahol pedig sikerülne, ott akár egy félig kiírt fájl tartalmát olvasnánk be, azaz kezelt hiba helyett csendben hibás statisztikát kapnánk.

Megjegyzés: A modell viselkedése egységteszttel is ellenőrizhető, valódi fájl nélkül: a `Moq` `Raise` metódusával kívülről kiválthatjuk a *mockolt* interfész eseményét:

```
_mock.Raise(fileManager => fileManager.ContentChanged += null, EventArgs.Empty);
```